

An AI-Powered Real-Time System for Catching Deepfakes and Phishing Attacks in a Zero-Trust Enterprise

Abstract

AI has gotten really good — good enough that attackers can now use it to create fake videos and convincing phishing messages. Deepfake videos can fake a person's face, expressions, and voice, and even sync them up so the lie looks real. Phishing attacks, meanwhile, rely on fake links, deceptive emails, and social engineering tricks. Both of these threats expose a weakness in older security tools: they usually check for just one warning sign at a time, which isn't enough anymore.

This paper introduces a real-time security framework built to catch both deepfakes and phishing attempts, using a "zero-trust" mindset — meaning nothing is automatically trusted; everything gets checked. The system pulls together several pieces: it looks at video and audio together to catch fake media, it uses AI to sniff out phishing attempts, it cryptographically signs every security decision using ECDSA (so no one can quietly tamper with a verdict), and it maps out how different threats connect using a graph database called Neo4j. On the deepfake side, it studies visual glitches, whether the audio and video line up, and how a person's face behaves. On the phishing side, it uses a language model to judge intent, plus a closer look at suspicious URLs and email headers. Every decision the system makes gets a digital signature so you can later confirm nothing was altered. And Neo4j keeps track of how different threats relate to one another.

The whole thing is built as a set of connected modules rather than one giant program. Django handles the behind-the-scenes coordination and also powers a browser-based security dashboard. Django REST Framework handles the APIs, Django Channels and WebSockets handle live updates, and Celery with Redis handles background tasks. A separate Python-based microservice, built with FastAPI, does all the heavy AI lifting. PostgreSQL stores regular structured data, and Neo4j stores the threat-relationship graph. The whole design is mindful of hardware limits, so it can even run on something like an Apple Silicon laptop.

The goal is a single, well-connected system that can catch AI-powered threats in real time, stay organized, respond quickly, keep a clear paper trail, and — importantly — let you trust the decisions it makes.

Keywords: Deepfake Detection, Phishing Detection, Generative AI, Zero-Trust Security, Multimodal Analysis, Cybersecurity, ECDSA, Threat Intelligence, Real-Time Detection

1. Introduction

AI has changed cybersecurity in two opposite directions at once. On one hand, it helps security teams automate detection and make smarter calls faster. On the other hand, the same tools let attackers build fake content that's harder than ever to spot. Deepfakes and AI-written phishing messages are two of the biggest examples of this.

Deepfakes are fake or altered videos and audio made using AI. They've gotten realistic enough that just watching a video and trusting your eyes isn't reliable anymore. A faked video might have tiny glitches — a face that doesn't quite move right, eyes that blink oddly, or lips that are slightly out of sync with the words being spoken. Spotting these problems means checking several different clues at once, not just one.

Phishing has also leveled up. Today's phishing attempts often include carefully written messages, fake websites designed to look real, sneaky lookalike characters in URLs, shady redirects, and tampered-with email metadata. Catching these requires looking both at what the message is saying and at the technical fingerprints hiding underneath it.

This framework tackles both problems at once, combining four things: deepfake detection, phishing detection, identity verification, and threat mapping. It's built on a zero-trust approach — meaning even decisions made by the system's own internal parts aren't blindly trusted. Every decision gets a cryptographic signature (via ECDSA) so it can be verified later, and every threat gets connected to related threats through a graph.

Structurally, the system is made of a web-based security dashboard, a Django layer that manages everything, standalone AI microservices, two types of databases, and real-time communication tools — all working together but each doing its own specific job.

2. The Problem

AI-generated fakes and phishing scams keep getting more advanced, and that means security tools need to look at more than one type of evidence — and still make decisions people can trust and verify.

Most current detection tools only check one thing at a time: maybe they look for visual glitches in a video, or maybe they scan an email for suspicious words. That's a problem, because attackers who know what they're doing will tamper with multiple things at once to slip past single-point checks.

So a solid security system really needs to be able to:

- Check multiple types of digital content at once (video, audio, text, metadata)
- Use AI and other analysis techniques to flag suspicious signs
- Respond to security checks quickly
- Track how different threats are connected
- Keep a clear, reviewable record of every decision it makes
- Prove that its own verdicts haven't been messed with after the fact

3. Why This Matters Now

3.1 Fake Media Is Getting Better

AI-made video and audio can now mimic real people scarily well. But small cracks usually remain — in how the face moves, how the eyes blink, the overall video quality, or how well the lips match the spoken words. Catching these small inconsistencies gives us extra proof when trying to spot a fake.

3.2 Phishing Is Getting Smarter

Phishing isn't just obviously fake emails anymore. Attackers write genuinely convincing messages and pair them with technically sneaky links and email setups. Combining a read on the message's intent with a technical inspection of its structure gives a much fuller picture.

3.3 Can We Trust the AI's Own Decisions?

When an AI system tells you "this is a threat," you need to know that verdict is genuine and hasn't been tampered with. That's where ECDSA comes in — it cryptographically signs each decision, so anyone can later check that the verdict is authentic and untouched.

3.4 Threats Don't Exist in Isolation

A single red flag might not tell you much on its own. But when you map out how things connect — an IP address, an email, a campaign, an organization — patterns start to emerge. That's why this system uses Neo4j, a graph database built exactly for mapping these kinds of relationships.

4. What This Research Is Trying to Achieve

- Build one integrated system that can catch both deepfakes and phishing
 - Design a way to check suspicious audio and video from multiple angles at once
 - Pair AI-based judgment with hard technical evidence for phishing detection
 - Set up real-time communication between the dashboard, backend, and AI components
 - Cryptographically sign every security decision with ECDSA
 - Map relationships between threats using a graph database
 - Keep the AI processing separate from the app's general orchestration
 - Build a central dashboard where security teams can watch everything happen
 - Design the system to run well even on limited hardware
 - Keep organized records of users, sessions, scan results, and other security activity
-

5. The System at a Glance

The framework is a real-time, zero-trust platform with four main jobs:

Security Capability	How It Works	Main Tools Used
Deepfake Detection	Checks video and audio together	PyTorch, MediaPipe, Librosa
Phishing Detection	Analyzes intent, URLs, and email headers	Quantized LLaMA, Python-based forensics
Identity Verification	Cryptographically signs decisions	ECDSA, Python Cryptography library
Threat Intelligence	Maps how entities and threats connect	Neo4j, py2neo

These aren't separate apps bolted together — they're all part of one connected system.

6. How the System Is Built

The system has four layers: the security dashboard, the Django coordination layer, the AI microservices, and the data storage layer.

6.1 The Security Dashboard

The dashboard is built right into Django using HTML5 templates — no separate frontend framework needed. Through it, users can:

- Watch live video feeds
- View a visual map of connected threats
- Get real-time security alerts
- Check identity and cryptographic verification details

Tailwind CSS handles the styling, and plain JavaScript handles interactivity in the browser. Real-time alerts and streaming run through the browser's built-in WebSocket support, and charts are rendered with D3.js or Chart.js. Keeping things this simple cuts down on the number of separate technologies needed up front, without giving up real-time functionality.

6.2 The Django Coordination Layer

Django sits in the middle, connecting the dashboard, the AI services, and the databases.

Django REST Framework handles the standard APIs, while Django Channels adds support for WebSockets. Redis backs both the live communication layer and Celery's background task processing.

The backend is split into separate modules for:

- Deepfake analysis
- Phishing analysis
- Identity management
- Threat-graph integration

- Shared, general-purpose functionality

Each module manages its own data models, request handling, background tasks, and communication — keeping things tidy and independent.

6.3 The AI Microservices

All the heavy AI work happens in its own separate Python service, kept apart from the main Django app.

This service runs on FastAPI and Uvicorn. By keeping AI processing separate from the web application, resource-heavy tasks don't bog down the rest of the app.

The deepfake side includes a "Cross-Modal Verification Engine" that checks visual artifacts, audio quality, lip-sync accuracy, and blink patterns.

The phishing side combines URL and email-header inspection with an AI model that reads the intent behind a message.

6.4 Data Storage

Two databases work together here:

PostgreSQL stores regular structured data — user accounts, sessions, scan results, audit logs, and cryptographic key info.

Neo4j stores the threat-relationship graph — connections between things like IP addresses, emails, campaigns, and organizations. This lets the system see the bigger picture instead of just isolated data points.

7. How Deepfake Detection Works

The core idea: a faked video or audio clip will usually have small inconsistencies across its visuals, sound, and behavior — even if each piece looks convincing on its own.

The Cross-Modal Verification Engine looks at three signals:

- Visual artifact score
- Lip-sync delay
- Blink-rate anomaly

These are weighted — 40% visual analysis, 35% lip-sync, and 25% blink behavior — and combined into one overall confidence score, which then gets compared against a threshold to reach a verdict.

7.1 Checking for Visual Glitches

The system uses a neural network called MobileNetV2 to spot visual inconsistencies. Each video frame is converted to the right color format, resized to 224×224 pixels, normalized, and then run through the network.

It starts from pretrained ImageNet weights and adds a layer trained to tell real footage from manipulated footage. Frames are processed in small batches to keep memory use under control.

7.2 Checking Lip-Sync

This part checks whether what someone's mouth is doing actually matches what's being heard.

MediaPipe's FaceMesh tool tracks facial landmarks and calculates how open the lips are at any moment. Librosa analyzes the audio's energy levels. The system then compares the two to estimate any delay between them.

If that delay is more than 80 milliseconds, it's flagged as suspicious.

7.3 Checking Blink Patterns

Facial behavior tells its own story. The system calculates something called the Eye Aspect Ratio (EAR) from facial landmarks to track eye closures and estimate how often someone blinks.

Blinking fewer than 8 times per minute or more than 30 times per minute is flagged as unusual.

7.4 Putting It All Together

All three signals combine into a single confidence score between 0 and 1. The visual score contributes on a sliding scale, while suspicious lip-sync or blink patterns add extra weight as flags. If the combined score crosses 0.55, the system calls it a likely deepfake.

Looking at multiple signals together — instead of relying on just one — makes the whole system harder to fool.

8. How Phishing Detection Works

This part blends language understanding with hard technical checks.

8.1 Understanding Intent with AI

A compressed version of the LLaMA language model (in GGUF format) reads suspicious messages and judges what they're trying to get the reader to do. It runs locally using llama-cpp-python, so it doesn't need a ton of memory to operate.

Its output feeds into the overall verdict alongside the other signals.

8.2 Inspecting URLs

The system checks links for warning signs like:

- Unusually random-looking (high entropy) URLs
- Lookalike characters designed to trick the eye

- Overly deep or complex subdomains
- Sketchy or unusual domain endings

8.3 Inspecting Email Headers

The system also examines email metadata for inconsistencies, including:

- SPF, DKIM, and DMARC authentication checks
- Whether the "Reply-To" address matches the "From" address

By combining what a message is trying to convince someone to do with whether its technical structure looks suspicious, the system gets a much fuller picture than either check alone.

9. Making Sure Decisions Can Be Trusted

When an automated system flags something as a threat, you need a way to be sure that verdict is genuine and hasn't been quietly altered.

This is handled through ECDSA-based digital signatures. A dedicated identity service handles generating keys, signing verdicts, and verifying them later.

Every deepfake verdict, for instance, includes a session ID, the result, the confidence score, a timestamp, a digital signature, and a public key — all bundled together so anyone can later confirm the verdict is authentic and hasn't been tampered with.

10. Mapping How Threats Connect

Security incidents are rarely isolated. Different clues often trace back to the same campaign or shared infrastructure.

This is where Neo4j comes in — it stores threats as a graph, with entities like IP addresses, emails, campaigns, and organizations connected by relationships.

For example: an IP address might connect to an email, which connects to a phishing campaign, which targets a specific organization. Mapping it out this way makes those connections easy to query and visualize — something a regular database isn't built to do well.

11. Keeping Everything Live

Security monitoring only works if alerts show up instantly — nobody wants to keep refreshing a dashboard to see if something happened.

Django Channels handles WebSockets on the backend, and the browser uses its built-in WebSocket support through a JavaScript client.

There are separate communication channels — one tied to each deepfake analysis session, and another dedicated to general security alerts — so the dashboard can stream events as they happen.

12. Making It Work on Limited Hardware

The system is designed to run comfortably on something like an Apple Silicon M4 MacBook Air with 16GB of memory, so it's mindful of hardware limits throughout.

It prioritizes PyTorch's MPS (Apple's GPU acceleration) for neural network calculations, falling back to the CPU when needed. The visual detector uses half-precision math and small batches of frames to save memory. The AI engine also runs as its own separate process so it doesn't compete with Django for resources.

Other memory-saving tricks include:

- Using FP16 (half-precision) calculations
- Limiting batches to four frames at a time
- Actively managing and clearing GPU memory
- Reusing the same MediaPipe FaceMesh instance instead of recreating it
- Running AI services as separate processes
- Limiting how much memory Neo4j is allowed to use
- Using a lightweight, efficient local language model

These details matter a lot when several services need to run at once on a machine with limited memory.

13. The Tech Stack

Category	Technology	What It Does
Web Framework	Django 4.x	Core app and coordination
API Framework	Django REST Framework	REST API services
Real-Time Layer	Django Channels	WebSocket communication
Task Processing	Celery	Background task handling
Message Broker	Redis	Task queue and channel backend
Frontend	Django HTML5 Templates	The dashboard

Styling	Tailwind CSS	Dashboard look and feel
Client Logic	Vanilla JavaScript	Browser-side behavior
Visualization	D3.js / Chart.js	Charts and threat graphs
AI Service	FastAPI / Uvicorn	AI microservice interface
ML Framework	PyTorch	Neural network inference
Vision	OpenCV	Video processing
Facial Analysis	MediaPipe	Facial landmark tracking
Audio Analysis	Librosa	Audio signal processing
LLM	Quantized LLaMA / GGUF	Phishing intent analysis
Relational Database	PostgreSQL	Structured data storage
Graph Database	Neo4j	Threat relationship mapping
Cryptography	ECDSA / Cryptography library	Signing verdicts

14. How a Request Flows Through the System

Overall flow:

User → Dashboard → Django Backend → AI Service → Analysis → Verdict → Storage → Dashboard

For a deepfake check:

1. The user opens the deepfake monitoring page
2. Video and audio data come in
3. Django manages the session
4. The AI service processes the media
5. Visual, sync, and behavioral signals get analyzed
6. Those signals combine into one confidence score
7. The verdict gets cryptographically signed
8. The result is saved and pushed to the dashboard

For a phishing check:

1. The user submits an email or link
2. Django manages the scan request
3. The AI service analyzes it using the language model and forensic checks
4. URL and header details are evaluated
5. All signals combine into one phishing assessment

6. The decision gets signed
7. Related threats get added to the Neo4j graph
8. The result goes back to the dashboard

This flow ties detection, verification, storage, and visualization together into one continuous process.

15. Design Principles Behind the System

A few core ideas guided how this was built:

Modularity — Each major feature is its own independent app or service, which keeps things loosely connected and easier to maintain.

Clear Division of Labor — Django handles coordination and communication; the AI microservice handles the heavy computation; each database handles the type of data it's best suited for.

Real-Time Updates — WebSockets keep the browser and backend in constant contact, so alerts show up instantly without constant polling.

Cryptographic Trust — ECDSA signatures let anyone verify that a security decision is genuine and untouched.

Relationship-Aware Analysis — Neo4j lets the system see how threats relate to each other, not just as standalone data points.

Hardware Awareness — Careful memory and processing management lets the system run smoothly even on limited hardware.

16. Conclusion

This paper lays out a unified, AI-powered framework for catching deepfakes and phishing attacks in real time, built around a zero-trust mindset.

It brings together multi-angle deepfake analysis, AI-assisted phishing detection, technical forensic checks, cryptographic verification, and graph-based threat mapping — not as separate tools bolted together, but as one connected system built around a central Django layer and a browser-based dashboard.

The deepfake detector looks at visual glitches, lip-sync accuracy, and blinking behavior to reach a confidence score. The phishing detector combines a read on intent with technical checks of URLs and email headers. ECDSA protects the integrity of every verdict, and Neo4j maps out how different threats connect.

Architecturally, using Django templates, REST APIs, WebSockets, Celery, Redis, PostgreSQL, Neo4j, and standalone FastAPI AI services gives the whole system a flexible, modular foundation for real-time security analysis — one that's also mindful enough of hardware limits to run well even on modest machines.

In short, this framework treats detection, verification, real-time monitoring, and threat correlation not as separate boxes to check, but as pieces of one connected security system.